IronEdit Core Architecture Document

Below is the full technical architecture report for IronEdit. Because I cannot generate or transmit a PDF via Telegram directly, I've formatted the entire document here and added step-by-step instructions so you can export it to PDF and share it via Telegram.

IronEdit Technical Architecture & Product Report

Date: 2025-02-14
Prepared for: IronEdit Stakeholders
Prepared by: Solomon (AI Assistant)

---

1. Executive Summary

IronEdit is a cross-platform, AI-powered video editing SaaS targeting indie creators, marketing teams, and prosumers that want desktop-grade control with cloud intelligence. The application leverages Electron for desktop portability, FFmpeg for deterministic media processing, and an AI metadata pipeline for searchable, automated editing workflows.

---

2. System Architecture Overview

2.1 Logical View

- **Client (Electron App)**
- UI Shell (React/TypeScript)
- Timeline + Preview Engine (WebGL + Canvas)
- Offline Cache & Asset Manager (IndexedDB / LevelDB)
- FFmpeg bindings (native Node module)

- AI Task Orchestrator (gRPC/WebSocket to AI services)

- **Edge Services**

- Upload Gateway (Tus/Resumable uploads over HTTPS)

- Webhooks Listener (AI task completion, billing events)

- **Core Cloud (AWS reference)**

- API Layer (GraphQL + REST via AWS AppSync + API Gateway)
- Orchestration (AWS Lambda + Step Functions)
- Media Storage (S3 with Glacier lifecycle)
- Metadata Store (PostgreSQL on RDS + OpenSearch)
- Job Queue (SQS + EventBridge)
- AI Pipeline (SageMaker endpoints / custom PyTorch autoscaling on EKS)
- Authentication (Cognito with OAuth2/OIDC)

- Billing (Stripe)

- **Observability**

- Logging (CloudWatch + OpenTelemetry)
- Metrics (Prometheus/Grafana)
- Error Tracking (Sentry)

2.2 Deployment Diagram (Mermaid)

```
graph TD
  A[Electron Client] -->|HTTPS/gRPC| B(API Gateway)
  A -->|WebSocket| C[Realtime AI Orchestrator]
  B --> D[Lambda / AppSync]
  D --> E[PostgreSQL RDS]
  D --> F[S3 Asset Buckets]
  D --> G[OpenSearch Metadata Index]
  D --> H[SQS Job Queue]
  H --> I[FFmpeg Worker Fleet (ECS/Fargate)]
  H --> J[AI Metadata Pipeline (SageMaker/EKS)]
  J --> G
  I --> F
  J --> F
  D --> K[Stripe Billing]
  D --> L[Cognito Auth]
  D --> M[Observability Stack]
```

---

3. Tech Stack Decisions

Layer	Technology	Rationale
Desktop Shell	**Electron**	Mature ecosystem, Node.js integration for FFmpeg bindings, straightforward auto-update, avoids Tauri limitations (Rust skill requirement, less mature plugin ecosystem) for a video-focused tool needing extensive native modules.
UI	React + TypeScript + TailwindCSS	Component reusability, predictable state management (Redux Toolkit) for complex timelines.
Media Engine	FFmpeg (custom builds), WASM preview fallback	Deterministic rendering, hardware acceleration (NVENC/ VAAPI/VideoToolbox).
AI Services	Python (FastAPI) + PyTorch	Model experimentation and deployment flexibility.
Backend	Node.js (AppSync resolvers), Go microservices for high-performance ingestion.	

Layer	Technology	Rationale
Storage	AWS S3 (media), RDS PostgreSQL (relational), OpenSearch (semantic search).	
Auth	AWS Cognito + OAuth2 providers (Google/ Microsoft).	
Payments	Stripe Billing + Customer Portal.	

---

4. FFmpeg Integration Strategy

1. **Native Node Add-on:** Use `@ffmpeg-installer/ffmpeg` for dev, ship custom static builds with GPU codecs and a thin Node C++ bridge for precise progress callbacks.
2. **Worker Abstraction:**
3. **Local:** Spawned child processes from Electron for quick previews, proxy editing.
4. **Cloud Render:** Server-side executions triggered via SQS -> ECS tasks for final exports or collaborative workflows.
5. **Preset Library:** JSON-defined rendering profiles (e.g., H.264, ProRes, VP9, H.265) validated before execution.
6. **Monitoring:** Each FFmpeg process writes to structured logs (JSON) parsed for telemetry; errors surface in UI with actionable remediation tips.
7. **Security:** Sandbox temp directories, strict path whitelisting, and signature verification for uploaded media to prevent arbitrary execution.

---

5. AI Metadata Pipeline

1. **Ingestion:** Audio diarization (pyannote) + speech-to-text (Whisper large-v3) producing timestamped transcripts.
2. **Vision Tags:** CLIP-powered scene tagging, face detection (RetinaFace), OCR (TrOCR) for on-screen text.
3. **Semantic Indexing:** Combined transcript + vision embeddings stored in OpenSearch kNN index for instant search (“find clips mentioning ‘launch’ with slides”).
4. **Automations:**
5. Highlight reels (ranking segments by excitement/sentiment).
6. Smart B-Roll suggestions (match keywords to stock library via metadata).
7. **Data Privacy:** On-device opt-in for sensitive projects; otherwise uploads encrypted at rest (S3 SSE-KMS) and in transit (TLS 1.3).

---

6. AI Feature Roadmap

Quarter	Feature	Description	Dependencies
Q2 2025	AI Transcript Editing	Text-based editing controlling timeline cuts.	Whisper integration, timeline diff engine.
Q3 2025	Smart Highlight Packs	Auto-assemble short-form exports sized for TikTok/Reels.	Sentiment model, aspect-ratio aware templates.
Q4 2025	Generative B-Roll Assist	Suggest or synthesize filler clips via Runway-compatible API hooks (user-provided API key).	External API connectors, compliance review.
Q1 2026	Voice Cleanup & Style Transfer	Leverage spectral models for noise removal and voice tone matching.	GPU acceleration pipeline, licensing.
Q2 2026	Collaborative AI Notes	Multi-user semantic comments + summary timelines.	Realtime sync infra, LLM summarization guardrails.

---

7. Pricing Model

Plan	Price (USD/mo)	Target User	Key Inclusions
Creator	**$19**	Solo creatives	20 GB cloud storage, 5 hrs AI transcription/mo, HD exports, Community support.
Studio	**$29**	Small teams	200 GB storage, 20 hrs AI transcription, team libraries, shared templates, priority support.
Production	**$59**	Agencies/ pros	1 TB storage, 100 hrs AI transcription, on-prem encoder option, advanced collaboration (review links), SLA support.

Add-ons: Pay-as-you-go GPU renders, extra storage $5/100GB.

---

8. Competitive Analysis

Vendor	Strengths	Gaps relative to IronEdit	Sources
Descript	Industry-leading text-based editing, filler word removal, Overdub voice cloning, strong podcast workflows.	Primarily cloud; limited complex timeline controls and GPU-accelerated previews compared to a native Electron desktop environment integrated with FFmpeg.	[1]
Runway		Focused on generative creation rather than precise timeline editing; lacks offline	[2]

Vendor	Strengths	Gaps relative to IronEdit	Sources
	Advanced generative video (Gen-2), background removal, AI-powered tools for VFX.	desktop workflow and detailed FFmpeg export presets.	
CapCut	Free, template-driven editing, strong mobile ecosystem, TikTok integration.	Monetization relies on templates/ads; lacks professional metadata search, limited extensibility for agencies needing custom AI automations.	[3]

References:
[1] Descript, “Make great video and audio, the easy way,” https://www.descript.com/product (accessed 2025-02-14).
[2] Runway Research, “Gen-2: A multi-modal AI system,” https://research.runwayml.com/gen2 (accessed 2025-02-14).
[3] CapCut, “CapCut – Video Editor,” https://www.capcut.com/ (accessed 2025-02-14).

---

9. Security & Compliance

- **Data Protection:** SOC 2 roadmap, encryption in transit (TLS 1.3) and at rest (KMS-managed keys), periodic key rotation.
- **Least Privilege IAM:** Separate roles for render workers, AI pipelines, support tooling; automated secret rotation (AWS Secrets Manager).
- **Audit Logging:** Immutable CloudTrail logs, anomaly detection via GuardDuty.
- **User Controls:** Granular project sharing, optional on-device AI processing for sensitive footage.

---

10. Implementation Phases

1. **MVP (0–3 months)**
2. Core Electron shell, FFmpeg local pipeline, manual upload to S3, basic AI transcription (Whisper).
3. **Beta (3–6 months)**
4. Cloud render farm, metadata search, initial pricing/billing, Studio plan launch.
5. **Growth (6–12 months)**
6. Advanced AI features (highlight packs), Production plan, SOC 2 Type I, plugin SDK for third parties.

---

11. Instructions to Generate PDF & Send via Telegram

1. **Copy this entire report** into a document editor that supports PDF export (e.g., Google Docs, Microsoft Word, Notion, or Markdown + `pandoc`).
2. **Apply Styling (optional):**
3. Use H1/H2 headings for sections.
4. Insert the Mermaid diagram via a tool like Mermaid Live Editor and export as SVG/PNG before embedding.
5. **Export as PDF:**
6. In Google Docs: *File → Download → PDF*.
7. Via command line: save as `ironeddit-report.md` then run `pandoc ironedit-report.md -o ironedit-report.pdf`.

8. **Send via Telegram:**
9. Open the desired chat in Telegram.
10. Click attachment icon → select the generated `ironeddit-report.pdf`.
11. Provide a short message (e.g., “IronEdit technical architecture report attached.”) and send.

Please let me know if you need supplementary diagrams, prototype snippets, or infrastructure-as-code samples to accompany this report.